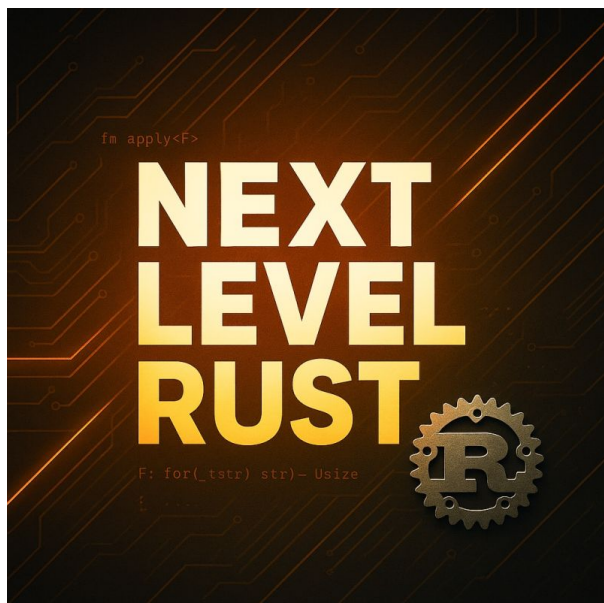




Copyrighted material

This is an integral part of course **Next level Rust** and cannot be used in any context without explicit permission from **CLEUTON SAMPAIO DE MELO JUNIOR** (a.k.a. "the Author"). You can't copy, change, publish, or share with others.

Advanced cargo usage

Chapter: Publishing and Maintaining Crates

In this chapter, we focus on how to publish a Rust crate to crates.io and maintain it using only Cargo commands and standard practices. You don't need external tools or continuous integration systems—everything can be managed directly through Cargo.

Preparing Your Crate

Before publishing, make sure your crate is ready for public use. This includes writing documentation, setting up metadata, and testing locally.

Start by configuring your `Cargo.toml` with essential information:

```
[package]
name = "my_crate"
version = "0.1.0"
description = "A lightweight text processing utility"
authors = ["Alice Developer <alice@example.com>"]
license = "MIT"
repository = "https://github.com/alice/my_crate"
edition = "2021"
```

Fields like `description`, `license`, and `repository` are required or strongly recommended when publishing. The `license` must match a standard SPDX identifier.

Writing Documentation

Use doc comments (`///`) to document public functions, structs, and modules. Include examples that Cargo can test.

Example:

```
/// Counts the number of words in a string.
///
/// # Example
/// ```
/// assert_eq!(my_crate::count_words("hello world"), 2);
/// ```
pub fn count_words(input: &str) -> usize {
    input.split_whitespace().count()
}
```

Generate and view documentation locally:

```
cargo doc --open
```

This builds HTML documentation and opens it in your browser. It helps verify that your comments are clear and properly formatted.

Testing Your Code

Ensure all functionality is covered by tests. Write unit tests and integration tests inside the `tests/` directory.

Run all tests before publishing:

```
cargo test
```

This checks that your code works as expected and that your examples are up to date.

Checking Code Quality

Format your code using:

```
cargo fmt
```

This ensures consistent style across your project. If you don't have `rustfmt`, install it via:

```
rustup component add rustfmt
```

Check for common mistakes and lints:

```
cargo clippy
```

Install `clippy` if needed:

```
rustup component add clippy
```

Fix any warnings before publishing.

Publishing to crates.io

First, create an account on crates.io and obtain an API token. Then log in via Cargo:

```
cargo login
```

Enter your token when prompted.

Now, verify that everything is in order:

```
cargo package
```

This command packages your crate and shows what will be uploaded, without actually publishing. It checks for missing metadata and files.

If no errors appear, publish your crate:

```
cargo publish
```

Your crate is now live on crates.io and can be added to other projects with:

```
[dependencies]  
my_crate = "0.1.0"
```

Updating Your Crate

To release a new version, update the **version** field in **Cargo.toml** following semantic versioning:

- **Patch update:** 0.1.0 → 0.1.1 (bug fixes)
- **Minor update:** 0.1.1 → 0.2.0 (new features, no breaking changes)
- **Major update:** 0.2.0 → 1.0.0 (breaking changes)

After updating the version, run:

```
cargo publish
```

Cargo will upload the new version.

Yanking a Version

If a published version has a critical issue, you can yank it:

```
cargo yank --version 0.1.0
```

Yanking prevents new projects from depending on that version, but existing builds remain unaffected. It does not delete the version.

To undo a yank:

```
cargo yank --version 0.1.0 --undo
```

Deprecating a Crate

Cargo does not support marking a crate as deprecated. However, you can publish a new version with a clear notice in the crate-level documentation:

```
//! **Deprecated**: This crate is no longer maintained.  
//! Consider using `new_crate` instead.
```

Update the README and description in **Cargo.toml** to reflect this.

Final Notes

You can fully manage a Rust crate using only Cargo. No external tools are required for publishing or maintenance. By using **cargo publish**, **cargo yank**, **cargo test**, **cargo fmt**, and **cargo clippy**, you ensure your crate is reliable, well-documented, and easy to maintain.

The Rust community values well-maintained crates. By following these practices, you contribute to a healthy ecosystem and make your work accessible and trustworthy.

I have a small published crate named `env-inspector` on `crates.io`:

https://crates.io/crates/env_inspector